

```

# =====

# RyuuGPT v3 FINAL TRAINER

# Phase 1: LM-only

# Phase 2: Reasoning-augmented

# =====


import os, sys, math, glob, random, logging, argparse

import time

import json

from contextlib import nullcontext


import numpy as np

import torch

import torch.optim as optim

from torch.utils.data import Dataset, DataLoader

from torch.nn.utils.rnn import pad_sequence

from torch.utils.tensorboard import SummaryWriter

from tqdm import tqdm


logger = logging.getLogger("ryuugpt_v3")


# -----

# Tokens

# -----

PAD_ID = 0

BOS_ID = 2

EOS_ID = 3


# -----

# Fix PyTorch 2.9 checkpoint warning

# -----

```

```

try:

    import torch.utils.checkpoint as _chk

    _orig = _chk.checkpoint

    def _ckpt_no_reentrant(fn, *a, **kw):

        kw.setdefault("use_reentrant", False)

        return _orig(fn, *a, **kw)

    _chk.checkpoint = _ckpt_no_reentrant

except Exception:

    pass


# -----
# Project root
# -----

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.append(ROOT)


from model.Ryuu_gpt import RyuuGPT
from model.config import RyuuGPTConfig


# -----
# CLI
# -----

parser = argparse.ArgumentParser("RyuuGPT v3 FINAL Trainer")


parser.add_argument("--data_dir", default="data/open/Ryuu_Developer_v1/tokenized")
parser.add_argument("--save_dir", default="checkpoints/v3_reasoning")
parser.add_argument("--log_dir", default="runs/v3_reasoning")
parser.add_argument("--tokenizer_path", default="tokenizer/bpe_tokenizer_postproc.json")
parser.add_argument("--vocab_size", type=int, default=30000)


parser.add_argument("--batch_size", type=int, default=4)

```

```
parser.add_argument("--grad_accum", type=int, default=8)
parser.add_argument("--max_steps", type=int, default=60000)
parser.add_argument("--eval_interval", type=int, default=1000)
parser.add_argument("--warmup_steps", type=int, default=5000)

parser.add_argument("--lr", type=float, default=3e-4)
parser.add_argument("--context_size", type=int, default=1024)

parser.add_argument("--n_layer", type=int, default=16)
parser.add_argument("--n_head", type=int, default=12)
parser.add_argument("--n_embd", type=int, default=768)
parser.add_argument("--dropout", type=float, default=0.1)

parser.add_argument("--enable_checkpointing", action="store_true")
parser.add_argument("--enable_reasoning_head", action="store_true")
parser.add_argument("--reasoning_loss_weight", type=float, default=0.1)

parser.add_argument("--use_bf16", action="store_true")
parser.add_argument("--seed", type=int, default=42)
parser.add_argument("--num_workers", type=int, default=(0 if os.name == "nt" else 2))
parser.add_argument("--pin_memory", action="store_true")
parser.add_argument("--persistent_workers", action="store_true")
parser.add_argument("--prefetch_factor", type=int, default=2)
parser.add_argument("--cudnn_benchmark", action="store_true")
parser.add_argument("--matmul_precision", type=str, default="high", choices=["highest", "high",
"medium"])
parser.add_argument("--compile", action="store_true")
parser.add_argument("--compile_mode", type=str, default="default")
parser.add_argument("--log_gpu", action="store_true")

# -----
```

```

# Dataset

# -----

class TokenizedDataset(Dataset):
    def __init__(self, data_dir, split, ctx):
        self.ctx = ctx

        files = sorted(f for f in os.listdir(data_dir) if f.startswith(split) and f.endswith(".bin"))

        self.bins = [os.path.join(data_dir, f) for f in files]

        self.idxs = [f.replace(".bin", ".idx") for f in self.bins]

        self._tokens = None

        self._indices = None

        self.counts = []

        for i in self.idxs:
            # int64 index entries

            if not os.path.exists(i):
                raise FileNotFoundError(f"Missing index file: {i}")

            size = os.path.getsize(i)

            self.counts.append(size // 8)

        self.cum = np.cumsum([0] + self.counts)

        logger.info(f"Loaded {sum(self.counts)} {split} samples")

    def _lazy_init(self):
        if self._tokens is not None:
            return

        self._tokens = [np.memmap(b, dtype=np.uint32, mode="r") for b in self.bins]

        self._indices = [np.memmap(i, dtype=np.int64, mode="r") for i in self.idxs]

    def __len__(self):
        return sum(self.counts)

    def __getitem__(self, i):

```

```

self._lazy_init()

s = np.searchsorted(self.cum, i, side="right") - 1
l = i - self.cum[s]

start = 0 if l == 0 else self._indices[s][l - 1]
end = self._indices[s][l]

seq = self._tokens[s][start:end][:self.ctx]

if len(seq) < 2:
    return torch.tensor([PAD_ID]), torch.tensor([PAD_ID])

return (
    torch.tensor(seq[:-1], dtype=torch.long),
    torch.tensor(seq[1:], dtype=torch.long),
)

```

```

def pad_collate(batch):
    xs, ys = zip(*batch)
    return (
        pad_sequence(xs, batch_first=True, padding_value=PAD_ID),
        pad_sequence(ys, batch_first=True, padding_value=PAD_ID),
    )

```

```

def main():
    args = parser.parse_args()

    def infer_vocab_size(tokenizer_path, fallback):
        try:
            with open(tokenizer_path, "r", encoding="utf-8") as f:
                obj = json.load(f)
            vocab = obj.get("model", {}).get("vocab")
            if isinstance(vocab, dict) and len(vocab) > 0:

```

```

        return len(vocab)

    except Exception:

        pass

    return fallback

# -----
# Logging
# -----

os.makedirs(args.save_dir, exist_ok=True)

os.makedirs(args.log_dir, exist_ok=True)

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s INFO %(message)s",
    datefmt="%H:%M:%S"
)

logger = logging.getLogger("ryuugpt_v3")

logger.info("Starting RyuugPT v3 FINAL trainer")

logger.info(f"Data dir: {args.data_dir}")

logger.info(f"num_workers={args.num_workers} pin_memory={args.pin_memory} "
            f"persistent_workers={args.persistent_workers} prefetch_factor={args.prefetch_factor}")

# -----
# Repro / device
# -----

random.seed(args.seed)

np.random.seed(args.seed)

torch.manual_seed(args.seed)

torch.cuda.manual_seed_all(args.seed)

```

```

device = "cuda" if torch.cuda.is_available() else "cpu"
logger.info(f"Device: {device}")

# -----
# TF32 (new API)
# -----

if device == "cuda":
    if args.matmul_precision == "highest":
        torch.backends.cuda.matmul.fp32_precision = "ieee"
        torch.backends.cudnn.conv.fp32_precision = "ieee"
    else:
        torch.backends.cuda.matmul.fp32_precision = "tf32"
        torch.backends.cudnn.conv.fp32_precision = "tf32"
    if args.cudnn_benchmark:
        torch.backends.cudnn.benchmark = True
        logger.info("cudnn.benchmark enabled")
    logger.info(f"cuda matmul fp32 precision: {torch.backends.cuda.matmul.fp32_precision}")

# -----
# Precision
# -----

supports_bf16 = device == "cuda" and torch.cuda.is_bf16_supported()
use_bf16 = args.use_bf16 and supports_bf16

if use_bf16:
    autocast_dtype = torch.bfloat16
    scaler = None
    logger.info("Using BF16")
elif device == "cuda":
    autocast_dtype = torch.float16
    scaler = torch.amp.GradScaler("cuda")

```

```

        logger.info("Using FP16")
    else:
        autocast_dtype = torch.float32
        scaler = None
        logger.info("Using FP32")

def autocast_ctx():
    if device == "cuda":
        return torch.autocast("cuda", autocast_dtype)
    return nullcontext()

# -----
# Model
# -----

cfg = RyuuGPTConfig(
    vocab_size=infer_vocab_size(args.tokenizer_path, args.vocab_size),
    context_size=args.context_size,
    n_layer=args.n_layer,
    n_head=args.n_head,
    n_embd=args.n_embd,
    dropout=args.dropout,
    pad_token_id=PAD_ID,
    bos_token_id=BOS_ID,
    eos_token_id=EOS_ID,
    use_reasoning_head=args.enable_reasoning_head,
)

model = RyuuGPT(cfg).to(device)
logger.info(f"Using vocab size: {cfg.vocab_size}")

if args.enable_checkpointing and hasattr(model, "enable_gradient_checkpointing"):

```



```

    model.enable_gradient_checkpointing(True)
elif args.enable_checkpointing and hasattr(model, "gradient_checkpointing"):
    model.gradient_checkpointing = True

if args.compile:
    try:
        model = torch.compile(model, mode=args.compile_mode)
        logger.info(f"torch.compile enabled (mode={args.compile_mode})")
    except Exception as e:
        logger.warning(f"torch.compile failed: {e}")

logger.info(f"Model params: {sum(p.numel() for p in model.parameters())/1e6:.2f}M")

# -----
# Optimizer & Scheduler
# -----
optimizer = optim.AdamW(model.parameters(), lr=args.lr, betas=(0.9, 0.95))

def lr_schedule(step):
    if step < args.warmup_steps:
        return step / max(1, args.warmup_steps)
    p = (step - args.warmup_steps) / max(1, args.max_steps - args.warmup_steps)
    return 0.5 * (1 + math.cos(math.pi * p))

scheduler = optim.lr_scheduler.LambdaLR(optimizer, lr_lambda=lr_schedule)

# -----
# EMA
# -----
class EMA:
    def __init__(self, model, decay=0.9999):

```

```

        self.decay = decay

        self.shadow = {n: p.detach().clone() for n, p in model.named_parameters()}

    @torch.no_grad()
    def update(self, model):
        for n, p in model.named_parameters():
            self.shadow[n].mul_(self.decay)
            self.shadow[n].add_(p.detach(), alpha=1 - self.decay)

ema = EMA(model)

# -----
# Checkpoints
# -----

def cleanup_checkpoints(step, keep=2000):
    for ck in glob.glob(os.path.join(args.save_dir, "ckpt_step*.pt")):
        try:
            s = int(ck.split("step")[-1].split(".")[0])
            if s < step - keep:
                os.remove(ck)
        except Exception:
            pass

def save_checkpoint(step, best_val, is_best=False):
    ckpt = {
        "step": step,
        "model": model.state_dict(),
        "optimizer": optimizer.state_dict(),
        "scheduler": scheduler.state_dict(),
        "best_val": best_val,
        "ema": ema.shadow,
    }

```

```

}

path = os.path.join(args.save_dir, f"ckpt_step{step}.pt")

torch.save(ckpt, path)

logger.info(f"Saved checkpoint: {path}")


if is_best:

    best_path = os.path.join(args.save_dir, "ckpt_best.pt")

    torch.save(ckpt, best_path)

    logger.info(f"NEW BEST saved (val={best_val:.6f})")


def _extract_state_dict(ckpt):

    if isinstance(ckpt, dict):

        for key in ("model", "model_state", "state_dict"):

            if key in ckpt and isinstance(ckpt[key], dict):

                return ckpt[key]

    return ckpt


def load_latest_checkpoint():

    ckpts = glob.glob(os.path.join(args.save_dir, "ckpt_step*.pt"))

    best_ckpt = os.path.join(args.save_dir, "ckpt_best.pt")

    if not ckpts and not os.path.exists(best_ckpt):

        return 0, float("inf")

    latest = max(ckpts, key=lambda x: int(x.split("step")[-1].split(".")[0])) if ckpts else best_ckpt

    data = torch.load(latest, map_location=device)

    model.load_state_dict(_extract_state_dict(data), strict=False)


    if isinstance(data, dict):

        if "optimizer" in data:

            optimizer.load_state_dict(data["optimizer"])

        elif "optimizer_state" in data:

```

```

optimizer.load_state_dict(data["optimizer_state"])

if "scheduler" in data:
    scheduler.load_state_dict(data["scheduler"])
elif "scheduler_state" in data:
    scheduler.load_state_dict(data["scheduler_state"])

ema.shadow = data.get("ema", data.get("ema_state", ema.shadow))
step = int(data.get("step", 0))
best_val = float(data.get("best_val", float("inf")))
else:
    step = 0
    best_val = float("inf")

logger.info(f"Loaded checkpoint: {latest}")
return step, best_val

# -----
# Data
# -----

train_ds = TokenizedDataset(args.data_dir, "train", args.context_size)
val_ds = TokenizedDataset(args.data_dir, "test", args.context_size)

_pw = args.persistent_workers and args.num_workers > 0
_dl_kwargs = dict(
    num_workers=args.num_workers,
    pin_memory=args.pin_memory,
    persistent_workers=_pw,
)
if args.num_workers > 0:
    _dl_kwargs["prefetch_factor"] = args.prefetch_factor

```

```

train_loader = DataLoader(
    train_ds,
    args.batch_size,
    shuffle=True,
    collate_fn=pad_collate,
    **_dl_kwargs,
)

val_loader = DataLoader(
    val_ds,
    args.batch_size,
    shuffle=False,
    collate_fn=pad_collate,
    **_dl_kwargs,
)

writer = SummaryWriter(args.log_dir)

# -----
# Evaluation
# -----

@torch.no_grad()
def evaluate(step):
    model.eval()

    total_loss, total_reason = 0.0, 0.0

    n = 0

    for xb, yb in val_loader:
        xb, yb = xb.to(device), yb.to(device)

        with autocast_ctx():
            _, loss, _, reasoning = model(xb, yb)

        total_loss += loss.item()

```

```

        # reasoning may be None or a dict with "loss"

        if reasoning is not None and isinstance(reasoning, dict):
            total_reason += _safe_reason_loss(reasoning.get("loss", 0.0))

        n += 1

    model.train()

    avg_loss = total_loss / max(1, n)
    avg_reason = total_reason / max(1, n)

    return avg_loss, avg_reason

# -----
# TRAIN LOOP (FINAL)
# -----

step, best_val = load_latest_checkpoint()
pbar = tqdm(total=args.max_steps, initial=step)

# keep an exponential moving average of the training loss for reporting
train_loss_ema = None
train_reason_ema = None
ema_alpha = 0.99 # decay for the running average

# token usage tracking between eval logs
last_log_step = step
last_log_time = time.time()
tokens_since_log = 0

model.train()

def _safe_reason_loss(val):
    if val is None:
        return 0.0

    if torch.is_tensor(val):

```

```
        return float(val.detach()).item())
    return float(val)
```

```
def _log_gpu(step):
```

```
    if not args.log_gpu or device != "cuda":
```

```
        return
```

```
    try:
```

```
        alloc = torch.cuda.memory_allocated() / (1024 ** 2)
```

```
        reserved = torch.cuda.memory_reserved() / (1024 ** 2)
```

```
        max_alloc = torch.cuda.max_memory_allocated() / (1024 ** 2)
```

```
        logger.info(f"GPU MB alloc={alloc:.0f} reserved={reserved:.0f} max_alloc={max_alloc:.0f}")
```

```
    except Exception:
```

```
        pass
```

```
while step < args.max_steps:
```

```
    for xb, yb in train_loader:
```

```
        xb = xb.to(device, non_blocking=args.pin_memory)
```

```
        yb = yb.to(device, non_blocking=args.pin_memory)
```

```
        # count non-pad target tokens for usage stats
```

```
        with torch.no_grad():
```

```
            tokens_since_log += (yb != PAD_ID).sum().item()
```

```
    model._current_step = step
```

```
    if model.reasoning_head is not None:
```

```
        model.reasoning_head._current_step = step
```

```
    use_reasoning = (
```

```
        model.reasoning_head is not None
```

```
        and step >= model.reasoning_head.warmup_steps
```

```
    )
```

```

with autocast_ctx():
    _, lm_loss, _, reasoning = model(xb, yb)

    # loss already includes reasoning loss inside model.forward()

    loss = lm_loss

# update EMA of raw LM loss before gradient accumulation scaling
cur_loss = lm_loss.item()

if train_loss_ema is None:
    train_loss_ema = cur_loss
else:
    train_loss_ema = train_loss_ema * ema_alpha + cur_loss * (1 - ema_alpha)

# also track reasoning loss if present
if reasoning is not None and isinstance(reasoning, dict):
    cur_reason = _safe_reason_loss(reasoning.get("loss", 0.0))

    if train_reason_ema is None:
        train_reason_ema = cur_reason
    else:
        train_reason_ema = train_reason_ema * ema_alpha + cur_reason * (1 - ema_alpha)

loss = loss / args.grad_accum

if scaler:
    scaler.scale(loss).backward()
else:
    loss.backward()

if (step + 1) % args.grad_accum == 0:
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

    if scaler:
        scaler.step(optimizer)

```



```

        scaler.update()
    else:
        optimizer.step()
    optimizer.zero_grad(set_to_none=True)
    scheduler.step()
    ema.update(model)

step += 1
pbar.update(1)

if step % args.eval_interval == 0:
    val_loss, val_reason = evaluate(step)
    _log_gpu(step)
    # token usage stats
    now = time.time()
    steps_since = max(1, step - last_log_step)
    dt = max(1e-6, now - last_log_time)
    tok_per_step = tokens_since_log / steps_since
    tok_per_sec = tokens_since_log / dt
    tokens_k = tokens_since_log / 1000.0
    last_log_step = step
    last_log_time = now
    tokens_since_log = 0
    # compute reporting metrics
    train_ema = train_loss_ema if train_loss_ema is not None else float('nan')
    train_ppl = math.exp(train_ema) if train_ema < 1000 else float('inf')
    val_ppl = math.exp(val_loss) if val_loss < 1000 else float('inf')
    val_entropy = val_loss
    delta = train_ema - val_loss
    reason_ema = train_reason_ema if train_reason_ema is not None else float('nan')

```

```
logger.info(  
    f"Step {step}: Train EMA {train_ema:.6f} | Val Loss {val_loss:.6f} | "  
    f"TrainPPL {train_ppl:.6f} | ValPPL {val_ppl:.6f} | ValEntropy {val_entropy:.6f} | "  
    f"Delta {delta:.6f} | Reason EMA {reason_ema:.6f} | ValReason {val_reason:.6f} | "  
    f"TokK {tokens_k:.1f} | Tok/Step {tok_per_step:.1f} | Tok/s {tok_per_sec:.1f} | "  
    f"Best val loss {best_val:.6f}"  
)
```

```
is_best = val_loss < best_val  
  
if is_best:  
    best_val = val_loss  
  
save_checkpoint(step, best_val, is_best)  
cleanup_checkpoints(step)
```

```
if step >= args.max_steps:  
    break
```

```
logger.info("Training finished")  
logger.info(f"Best val loss: {best_val:.6f}")  
writer.close()  
pbar.close()
```

```
if __name__ == "__main__":  
    if os.name == "nt":  
        import torch.multiprocessing as mp  
        mp.freeze_support()  
    main()
```